

deep theory of algorithms even when the ease of importing a Python module makes such study seem unnecessary; it can always make you a better academic or practitioner.

This is perhaps the simplest possible chatbot, and it answers only questions related to chapters in this book. You could add so many enhancements to improve it: make the chapter descriptions more specific and thus more likely to match a broad range of queries; find a vectorization method that performs better than TFIDF; add more documents so that it could answer more queries. But although our chatbot is not the most advanced, we can be proud of it because it's ours and because we built it ourselves. If you can comfortably build a chatbot, you can consider yourself a competent designer and implementer of algorithms—congratulations for this culminating achievement in your journey through this book.

Becoming Better and Faster

You can do more with algorithms than you could when you started the book. But every serious adventurer will also want to be able to do things better and faster.

Many things can make you better at designing and implementing algorithms. Think about how each algorithm we implemented in this book relied on some understanding of a non-algorithmic topic. Our baseball-catching algorithm relies on an understanding of physics and even a little psychology. Russian peasant multiplication relies on an understanding of exponents and on deep properties of arithmetic, including binary notation. Chapter 7's geometry algorithms rely on insights into how points, lines, and triangles relate and fit together. The deeper your understanding of the field you're trying to write algorithms for, the easier it will be for you to design and implement algorithms. Thus, the way to get better at algorithms is easy: just understand everything perfectly.

Another natural next step for a budding algorithmic adventurer is to polish and repolish your raw programming skills. Remember that Chapter 8 introduced list comprehensions as a Pythonic tool that enables us to write language algorithms that are concise and perform well. As you learn more programming languages and master their features, you'll be able to write